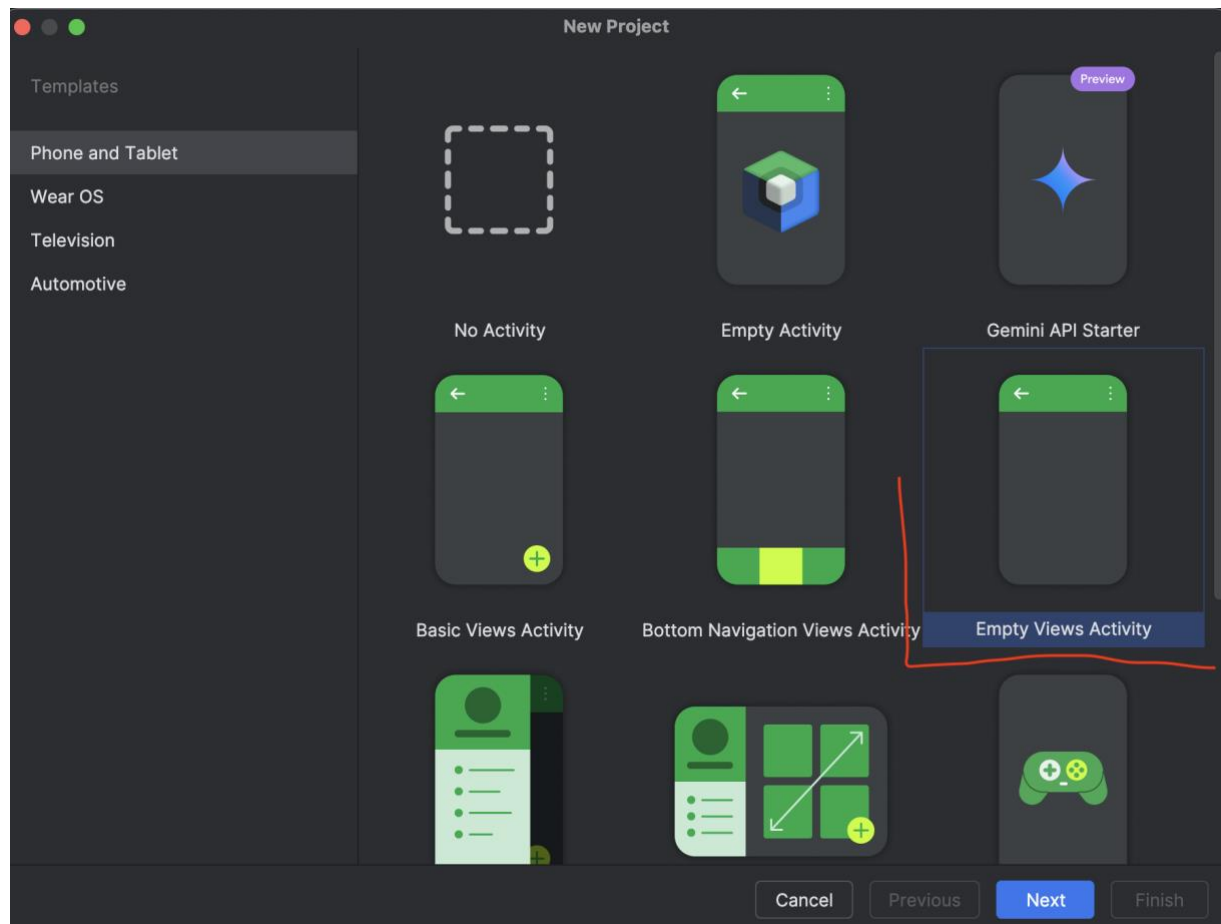
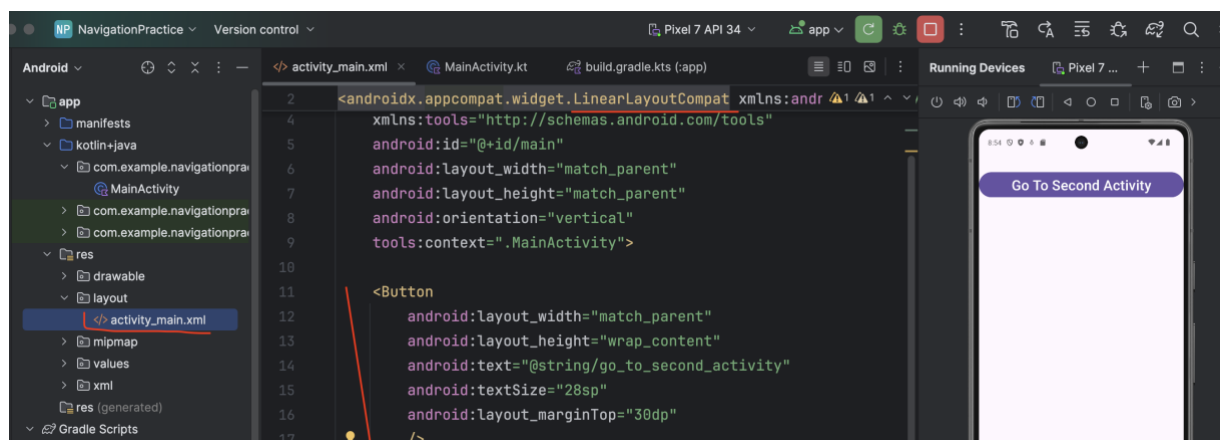


Android Apps

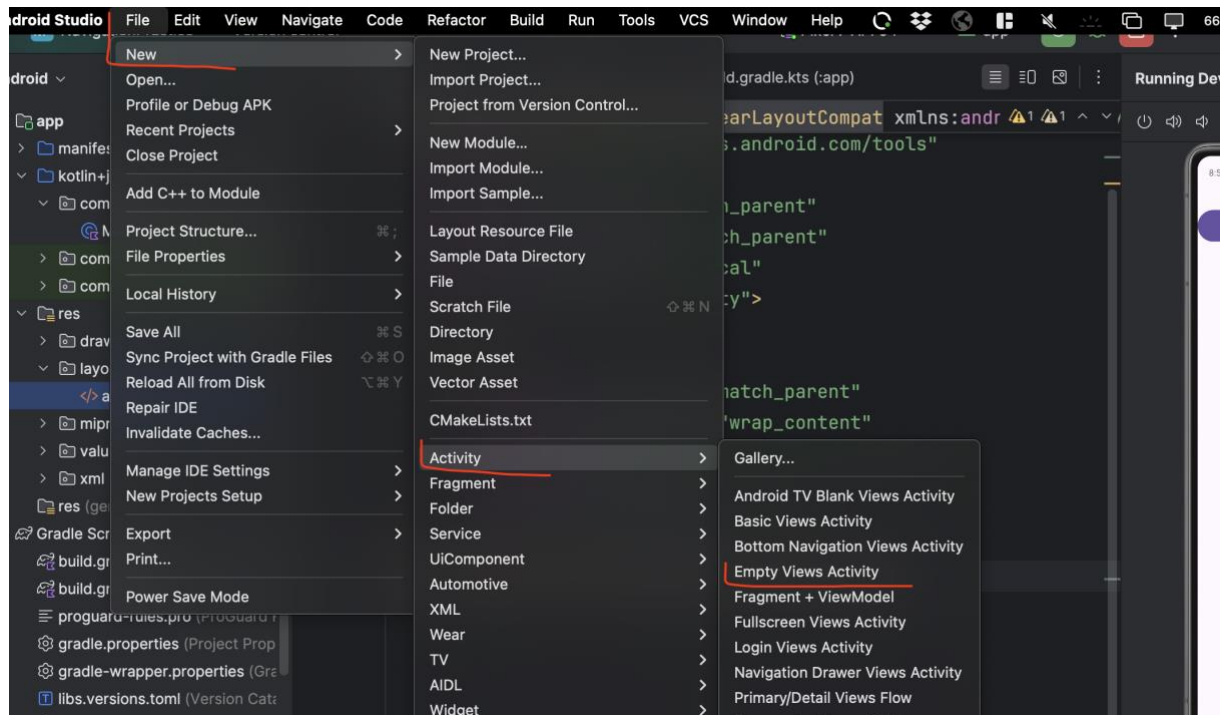
- Let's learn and practice now how to navigate between activities and fragments in Android APP.
- So, let's create a new Empty **View** Activity App (pls pay attention, it is not the same as Empty Activity, see the screen below), for API 34 and let's call it NavigationPractice



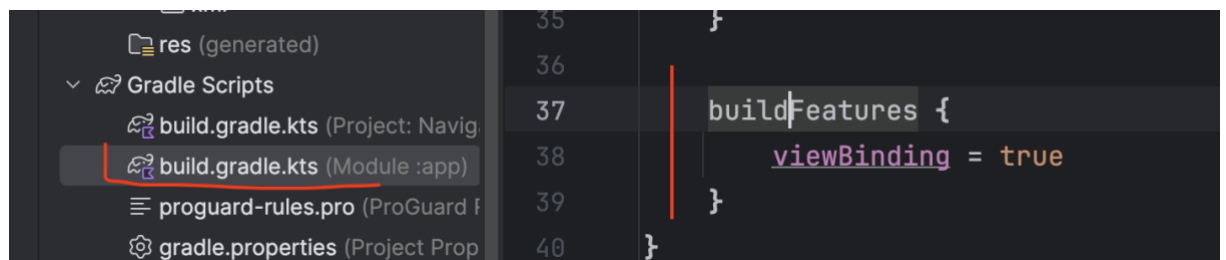
- Let's replace the main layout in our activity_main.xml with LinearLayoutCompat and let's add there a simple Button as it is shown below:



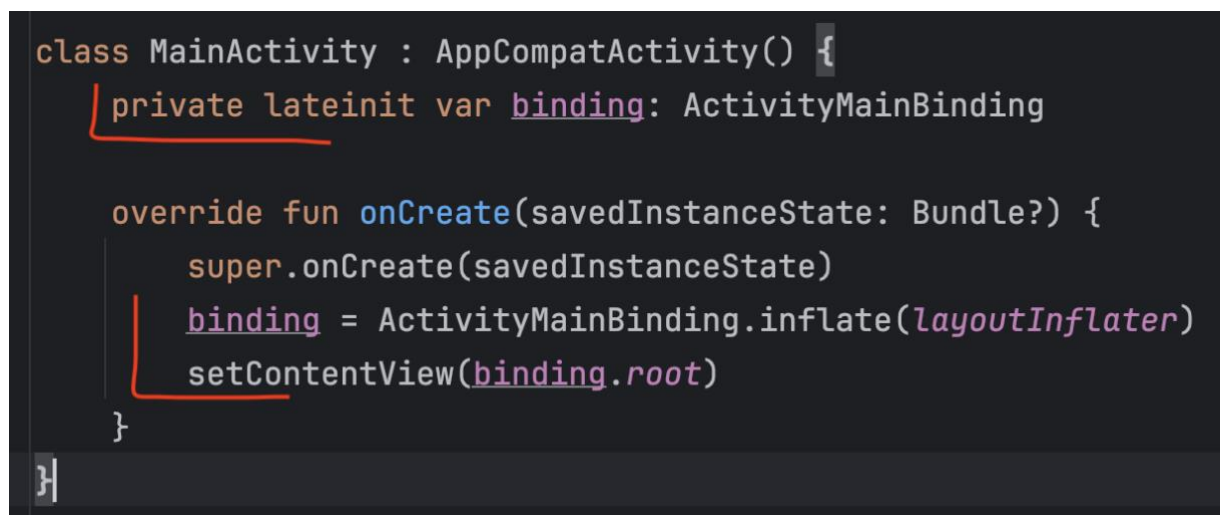
- Now, let's add a new Empty Views Activity to our project, and let's call it SecondActivity



- Now, let's turn on the view binding mechanism in our gradle file, as we did it on the previous class:v



- And then let's add and use the binding in MainActivity class:



- Just in case, let's run the app to verify that there are no any issues
-

- Let's go back to our activity_main.xml file and let's set the id property for our button there. And let's call it nav_to_sec_activity_btn
- Next, let's create the nav_to_sec_activity_btn onClickListener and let's try to navigate from MainActivity to SecondActivity.
- In the first step we will do it in an "old-fashioned" way i.e. using Intents and startActivity method. So let's make the implementation of our nav_to_sec_activity_btn onClickListener this way:

```

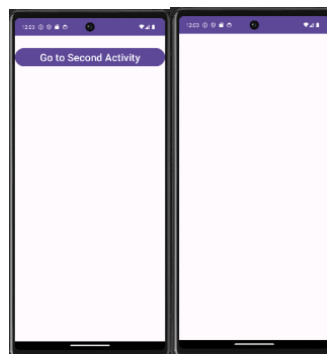
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)

    setNavToSecBtnOnClickListener()
}

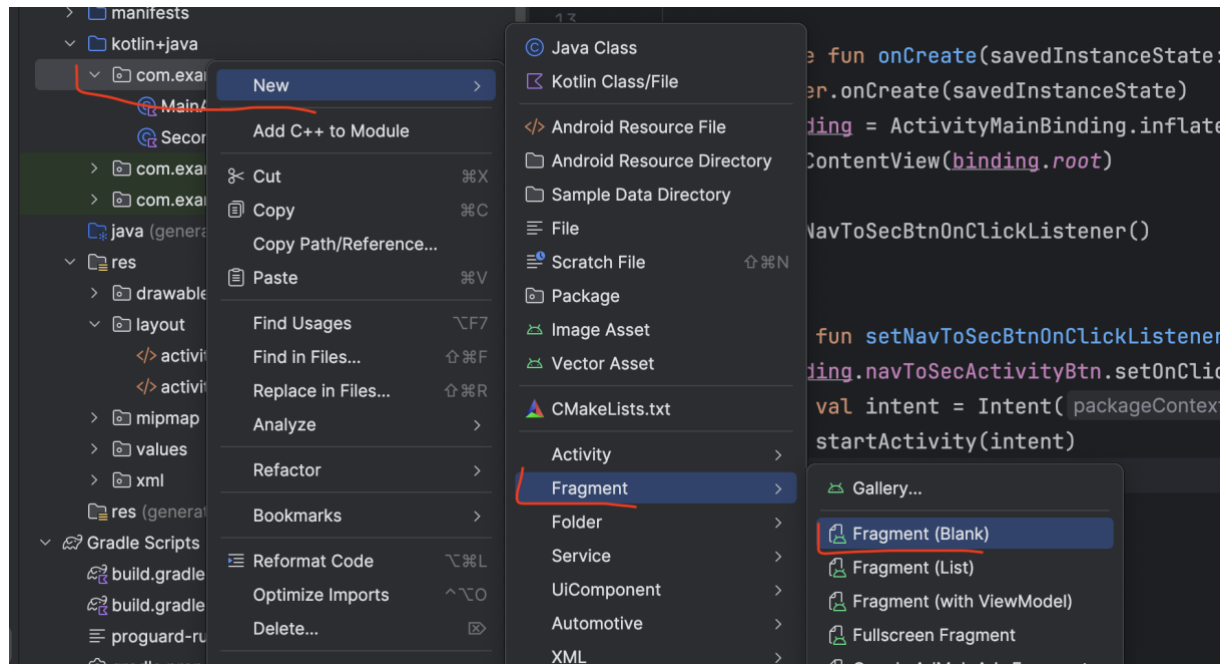
private fun setNavToSecBtnOnClickListener() {
    binding.navToSecActivityBtn.setOnClickListener {
        val intent = Intent(packageContext: this, SecondActivity::class.java)
        startActivity(intent)
    }
}

```

- And the navigation between the main and the second activity should work now. I.e. when you click on the "Go to Second Activity" the second activity should appear



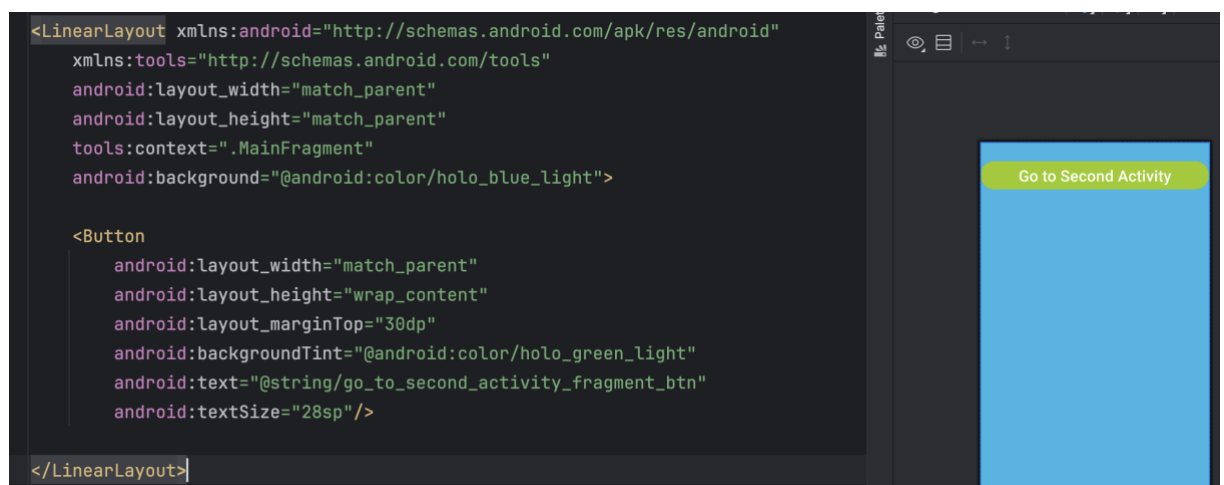
- Now let's define the navigation in a more up-to-date way i.e. using Fragments and Navigation Component.
- To make it work we need at least one fragment in our app. So let's add it by File->new as it is shown below, and let's call it MainFragment



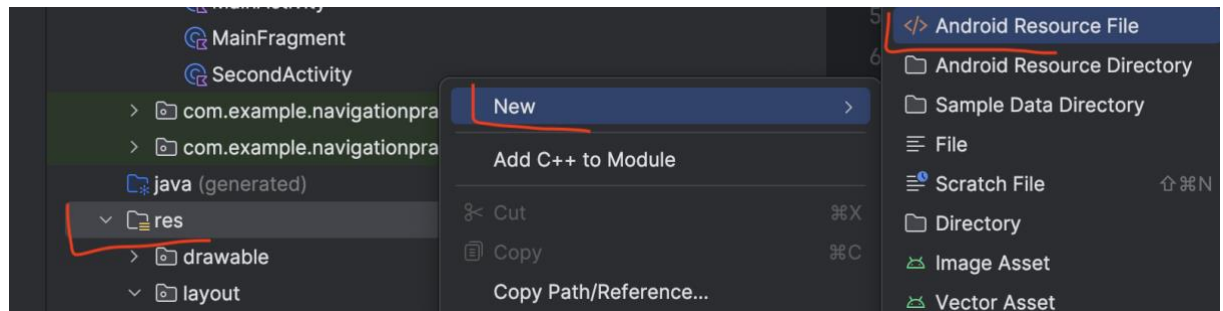
- In the MainActivity class let's remove all but onCreate method



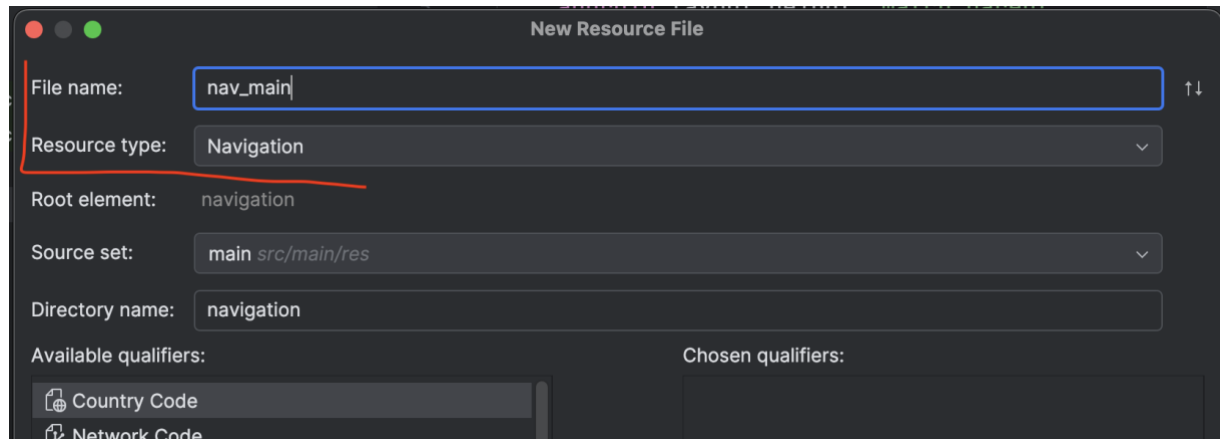
- Let's prepare the fragment_main layout by changing the layout type to the Linear one, adding a background color and a button as it is done below:



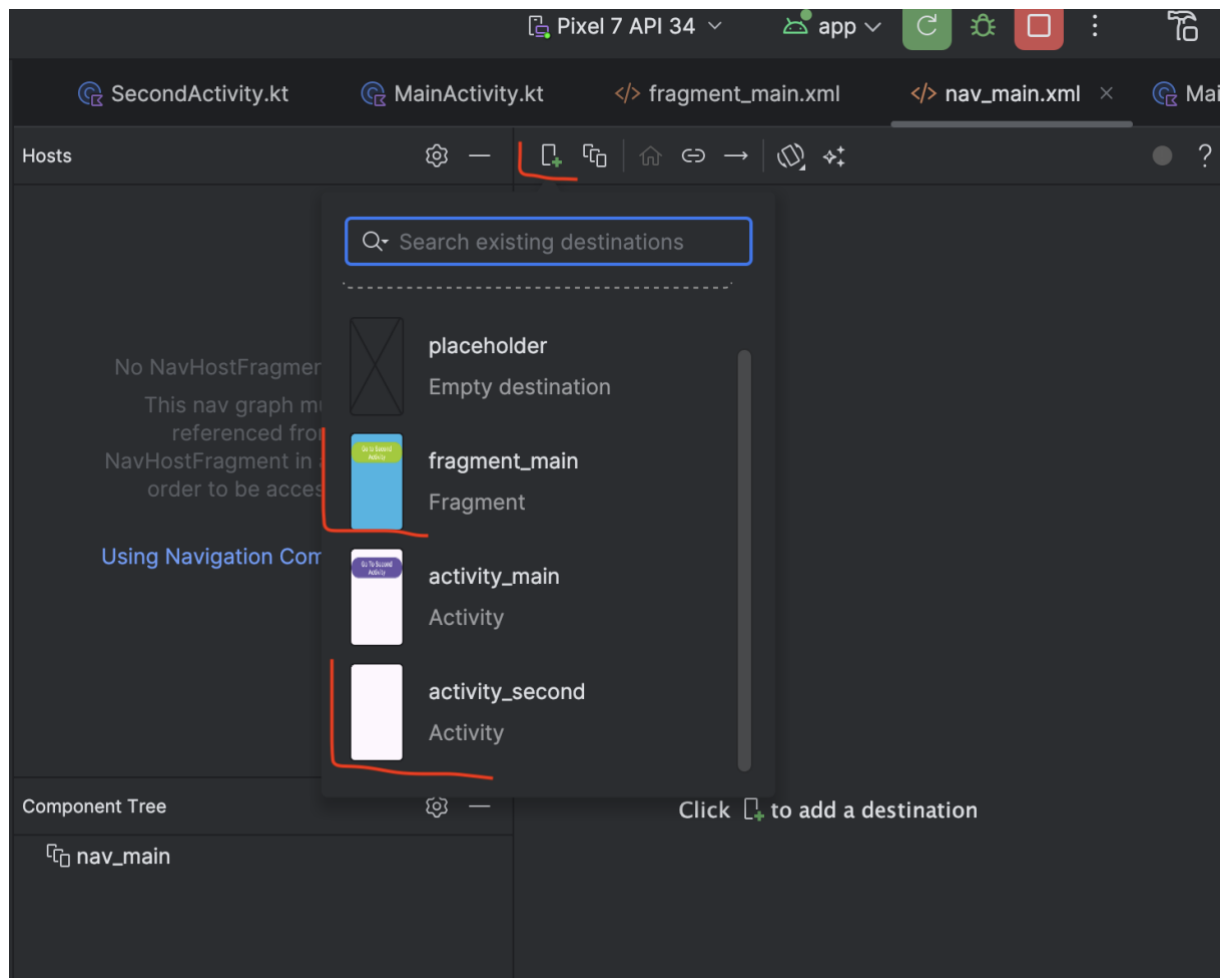
- Now let's add the navigation resource to our project. Then, let's right click on the res and then new -> Android Resource File



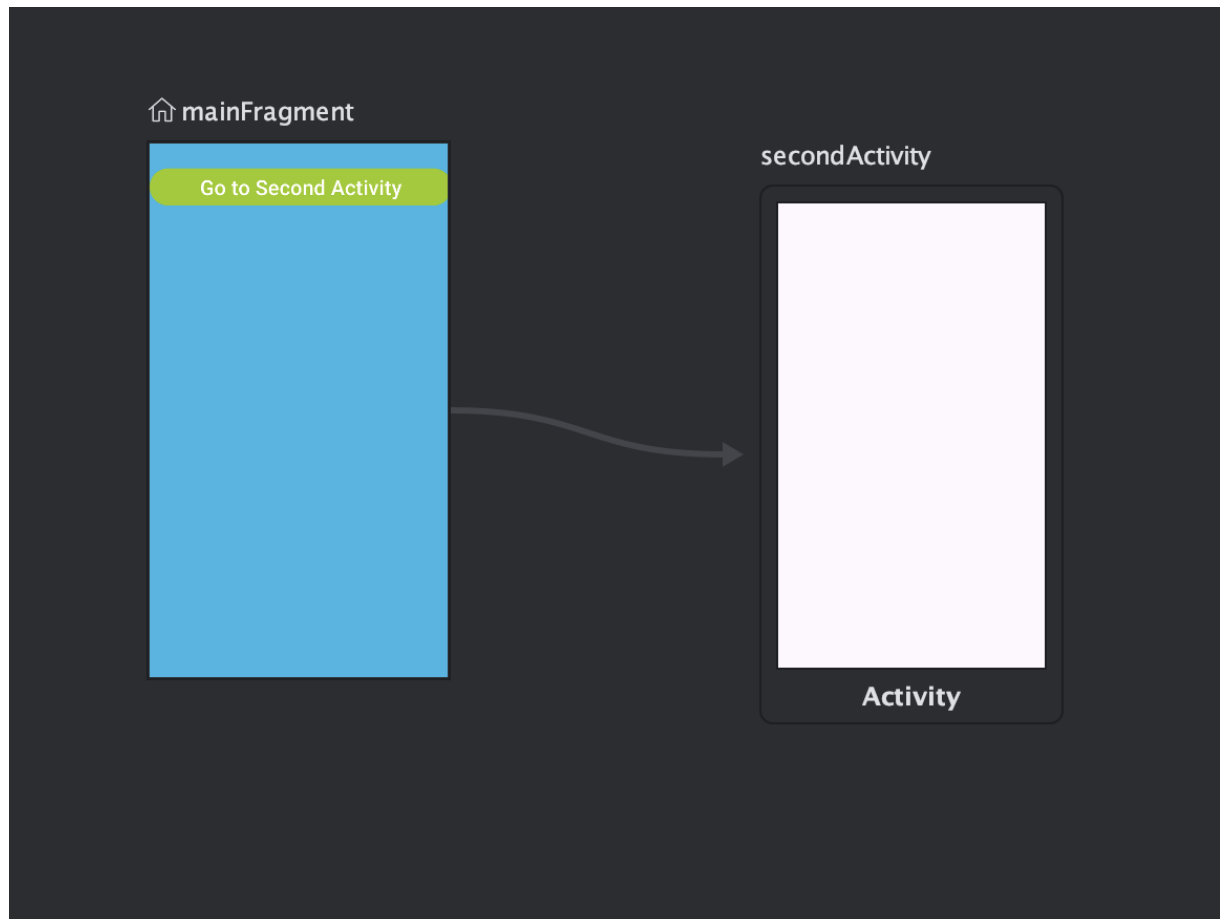
-
- And in the dialog box let's choose the navigation as type and nav_main as the name:



-
- If Android Studio is asking for adding some missing dependencies obviously let's agree.
- And now, let's add our fragment_main and activity_second as destinations to our navigation graph:



-
- And let's define the transitions between the mainFragment and the second activity as it is shown below:



-
- The nav_main navigation graph needs to be hosted by the activity which we want to make a starting point to our navigation (in our case it is MainActivity). So we need to add NavigationHost to our activity_main layout file as it is shown below:
-

- We may do it either directly in the main_activity.xml file by adding the following element:

```

9      tools:context=".MainActivity">
10
11      <Button
12          android:id="@+id/nav_to_sec_activity_btn"
13          android:layout_width="match_parent"
14          android:layout_height="wrap_content"
15          android:text="@string/go_to_second_activity"
16          android:textSize="28sp"
17          android:layout_marginTop="30dp"
18      />
19
20      <androidx.fragment.app.FragmentContainerView
21          android:id="@+id/nav_host_fragment_container"
22          android:name="androidx.navigation.fragment.NavHostFragment"
23          android:layout_width="match_parent"
24          android:layout_height="match_parent"
25          app:navGraph="@navigation/nav_main"
26      />
27
28  </androidx.appcompat.widget.LinearLayoutCompat>
  
```

The image shows an IDE with a project explorer on the left and a code editor on the right. The project explorer shows the file structure with `activity_main.xml` selected under the `layout` directory. The code editor displays the XML code for `activity_main.xml`, showing a button and a `FragmentContainerView` for navigation.

- The last thing that we need to do to make our navigation working is the implementation the action that should be done when we click on our green button
- So, first, let's add view binding mechanism in our MainFragment class in (almost) the same way as we did it before:

```
class MainFragment : Fragment() {
    private lateinit var binding: FragmentMainBinding
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        binding = FragmentMainBinding.inflate(inflater, container, attachToParent: false)
        return binding.root
    }
}
```

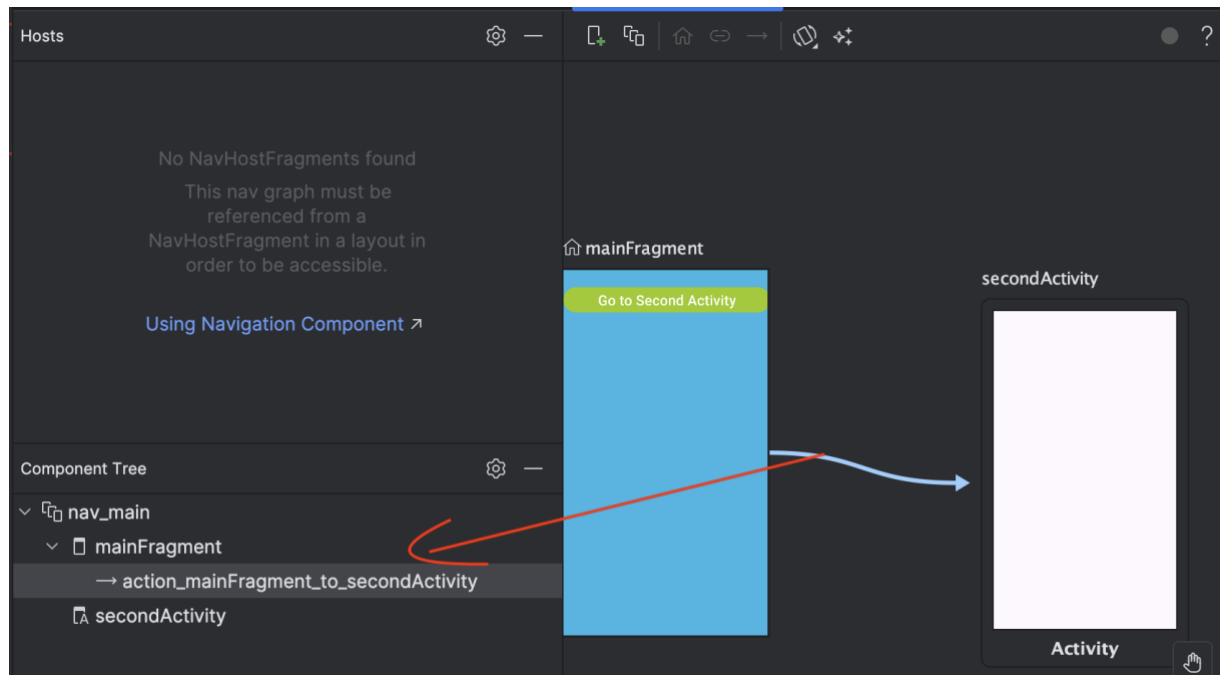
- Now, let's add the id to our green button e.g. like nav_to_sec_activity_from_fragment_btn
- And finally let's override the onViewCreated method in our MainFragment class as it is shown below:

```
class MainFragment : Fragment() {
    private lateinit var binding: FragmentMainBinding
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        binding = FragmentMainBinding.inflate(inflater, container, attachToParent: false)
        return binding.root
    }

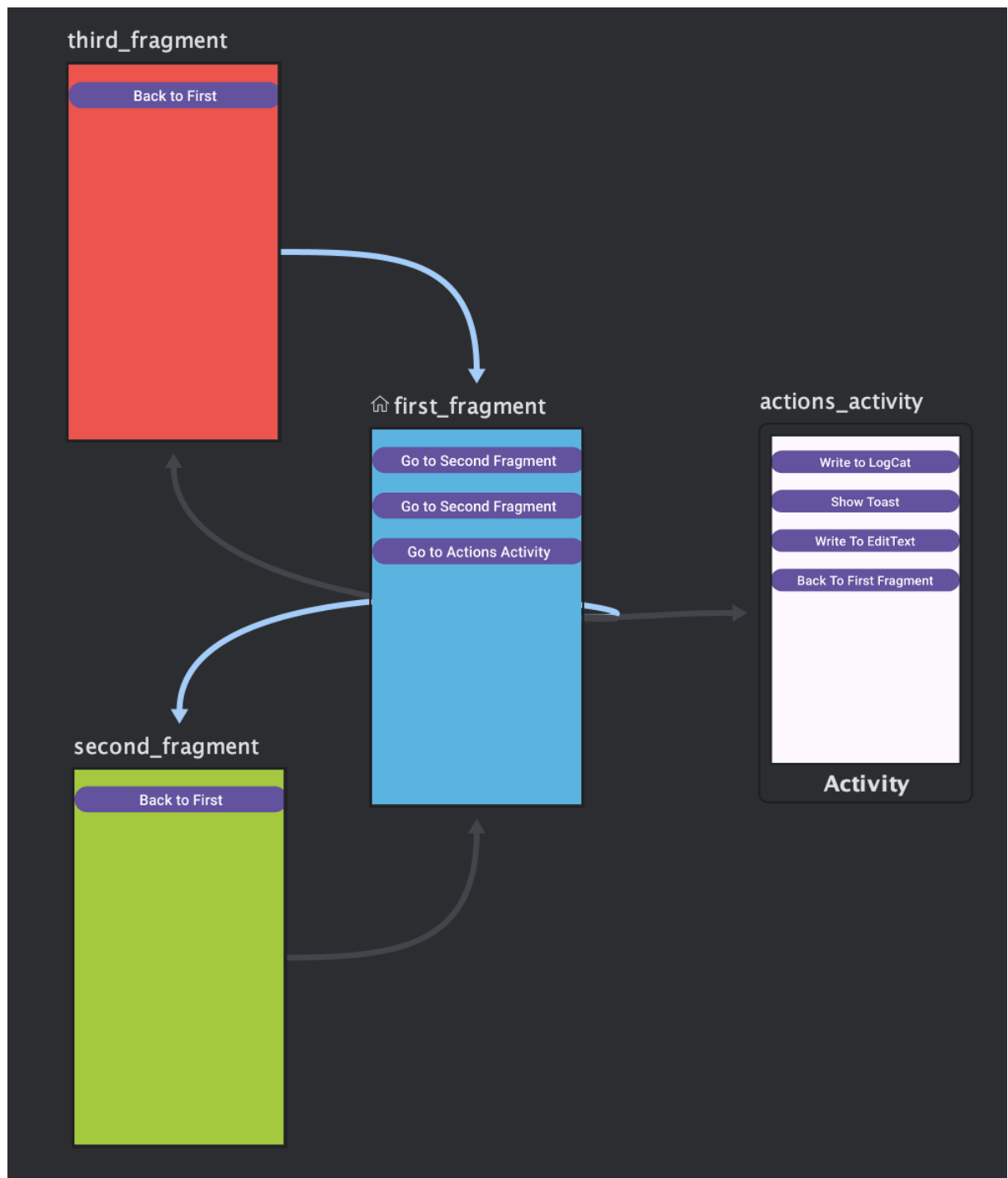
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        binding.navToSecActivityFromFragmentBtn.setOnClickListener {
            findNavController().navigate(R.id.)
        }
    }
}
```

main Int
 nav_main Int
 mainFragment Int
 action_mainFragment_to_secondActivity Int
 secondActivity Int
 nav_to_sec_activity_btn Int

- If you are not sure which id/action you should use, you may verify it in your Navigation Graph (nav_main.xml in our case) and check what is the id of the transition that you would like to use



-
- And from now on also our new navigation should work
- For practicing please implement the following navigation schema for your app:



-
- When done please upload the screenshots of your code and app to the UPEL platform